IMT Nord Europe – CERI Systèmes Numériques

Compte rendu de TP

Internet des Objets & Réseaux

Conception d'une application IoT de relevé
de température et d'humidité

Yassine EL FERGOUCH
Poste n° 42920

Année 2026

Table des matières

Introduction **2**

1 TP1 – La couche objets : simulation sous SimulIDE **2**

1.1 Prise en main de SimulIDE . . . 2
1.2 Mon premier circuit : clignotement d'une LED . . . 2
1.3 Mise en œuvre d'un bargraphe . . . 3
1.4 Mise en œuvre du capteur DHT22 . . . 3
1.5 Mise en œuvre d'un LCD 16x2 I2C . . . 4
1.6 LCD 16x2 I2C et capteur DHT22 . . . 4
1.7 Réalisation du capteur IdO « TempHumiX » . . . 5
1.8 Question : Manchester et Manchester différentiel . . . 6

2 TP2 – La couche réseau : Wi-Fi et protocole MQTT **7**

2.1 Paramètres utilisés . . . 7
2.2 Prise en main avec MQTTX . . . 7
2.3 Programmation de l'Arduino UNO R4 . . . 7
2.4 Amélioration de la communication . . . 9
2.5 Analyse des trames avec Wireshark . . . 10
2.6 QoS, messages retenus et jokers . . . 11
2.7 Questions générales . . . 12

3 TP3 – La couche application : IHM Python **13**

3.1 Client MQTT en console . . . 13
3.2 Thread MQTT . . . 14
3.3 IHM PySide6 – notice de fonctionnement . . . 15
3.4 Archivage SQLite . . . 15
3.5 Affichage des courbes . . . 16

4 Pour aller plus loin : interface web **16**

Conclusion **19**

Introduction

Le projet consiste à construire, étape par étape, une chaîne IoT complète autour d'un capteur de température et d'humidité (boîtier *TempHumiX*). Chaque TP correspond à un niveau de l'architecture vue en cours :

TP	Niveau de l'architecture	Contenu
TP1	Niveau 1 – Objets	Simulation SimulIDE : LED, bargraphe, DHT22, LCD I2C
TP2	Niveau 2 – Réseau	Arduino UNO R4 en Wi-Fi, NTP, MQTT, analyse Wireshark
TP3	Niveau 3 – Application	Client Python, IHM PySide6, base SQLite, courbes

Le capteur porte l'identifiant `42920` (numéro du poste). Tous les topics MQTT sont construits sur ce numéro, par exemple `/iot/TEMPERATURE_42920`.

1 TP1 – La couche objets : simulation sous SimulIDE

Lien avec le cours : niveau 1 de l'architecture IdO (capteurs / actionneurs / unité de traitement). Le microcontrôleur (ATmega328 de l'Arduino UNO) lit un capteur (DHT22) et commande des actionneurs (LED, bargraphe, afficheur). On reprend l'exemple du cours « allumage d'une LED » puis on ajoute un composant à la fois.

1.1 Prise en main de SimulIDE

SimulIDE réunit dans une seule fenêtre :

- la **bibliothèque de composants** (sources, interrupteurs, LED, capteurs, Arduino…) ;
- l'**éditeur de circuit**, où l'on place et relie les composants ;
- l'**éditeur de code**, où l'on écrit le croquis Arduino (`.ino`) ;
- la **fenêtre de sortie**, qui affiche les messages de compilation.

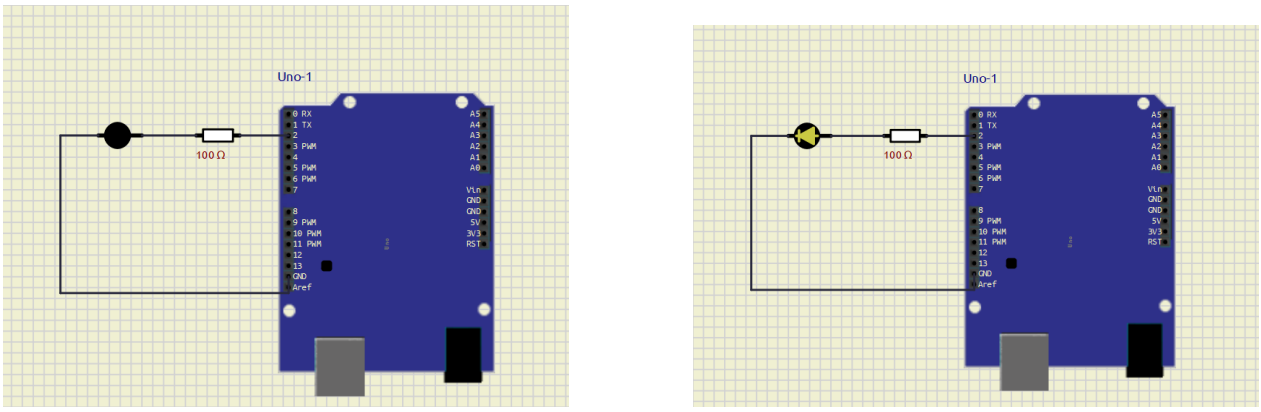
Cycle de travail pour chaque montage :

1. dessiner le circuit et l'enregistrer (`.sim1`) ;
2. écrire le code, puis *Lancer la compilation* (compilateur Arduino paramétré dans SimulIDE) ;
3. *Charger le code compilé* (`.hex`) dans l'Arduino UNO du circuit ;
4. *Lancer la simulation*, la mettre en pause si besoin pour observer un état.

Règle de nommage : le sous-répertoire, le fichier de simulation `.sim1` et le fichier de code `.ino` portent le même nom, en minuscules non accentuées avec `_`. Exemple : `capteur_dht22/capteur_dht22.sim1` et `capteur_dht22/capteur_dht22.ino`.

1.2 Mon premier circuit : clignotement d'une LED

Arduino UNO + LED + résistance de 100 Ω sur la broche 2. La résistance limite le courant dans la LED. `digitalWrite(HIGH)` met 5 V sur la broche (LED allumée), `LOW` met 0 V (LED éteinte), avec `delay(500)` entre les deux : la LED clignote à 1 Hz.



(a) LED éteinte (état 0) (b) LED allumée (état 1)

FIGURE 1 – `clignotement_led` : LED sur la broche 2

Une fois ce premier dossier créé, le dépôt Git/GitHub a été initialisé : un commit par montage, avec un message clair.

1.3 Mise en œuvre d'un bargraphe

Le « Led Bar » et le réseau de résistances DIP sont passés de 8 à 10 éléments (propriété *Size*). Les 10 LED sont reliées aux broches 3 à 12. Le code réalise un **chenillard** : une seule LED allumée, qui se décale d'une position toutes les 100 ms, de gauche à droite. Les broches sont rangées dans un tableau, ce qui permet de les parcourir avec une boucle `for`. Ce bargraphe sera piloté à distance par MQTT au TP2.

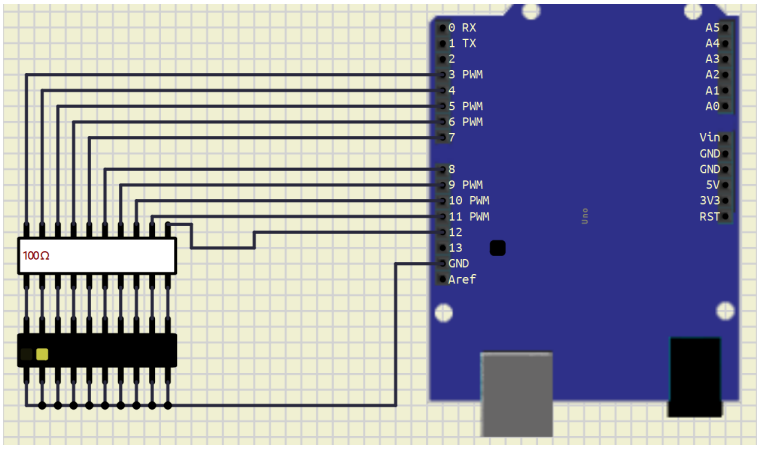
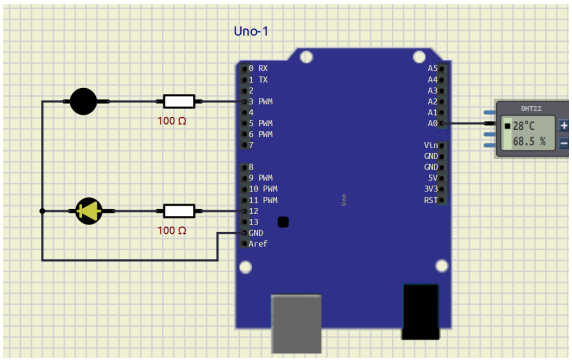


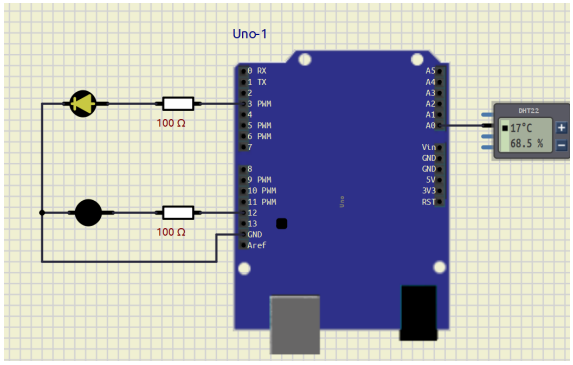
FIGURE 2 – `bargraphe` : 10 LED sur les broches 3 à 12

1.4 Mise en œuvre du capteur DHT22

Le DHT22 est branché sur `A0` et lu avec la bibliothèque *DHT sensor library* d'Adafruit (qui installe aussi *Adafruit Unified Sensor*). Le programme lit la température toutes les 2 s : LED verte (broche 3) si $T < 20\,°\text{C}$, sinon LED rouge (broche 12). On fait varier la température avec les boutons +/− du capteur simulé.



(a) $T = 28\,°C$: LED rouge



(b) $T = 17\,°C$: LED verte

FIGURE 3 – `capteur_dht22` : comparaison à un seuil de 20 °C

1.5 Mise en œuvre d'un LCD 16x2 I2C

Afficheur Aip31068 relié au bus I2C : SDA sur `A4`, SCL sur `A5`, adresse `0x3E`. Bibliothèque *LiquidCrystal AIP31068*. Attention : le fichier à inclure est `LiquidCrystal_AIP31068_I2C.h` et non `LiquidCrystal_I2C.h`. `lcd.setCursor(colonne, ligne)` place le texte : « HELLO » en (0,0), « WORLD » en (7,1).

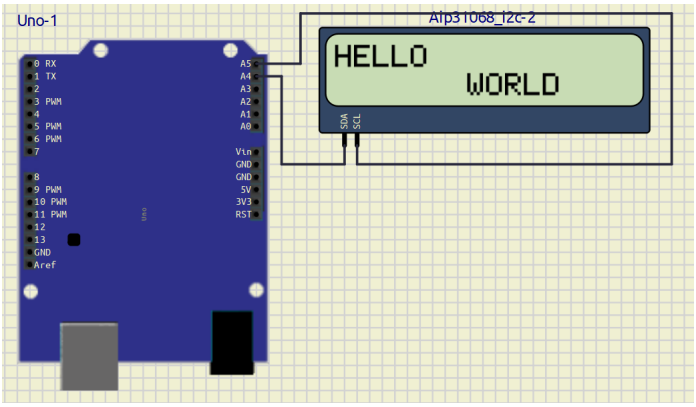


FIGURE 4 – `lcd_i2c` : affichage « HELLO WORLD »

1.6 LCD 16x2 I2C et capteur DHT22

On réunit les deux montages précédents : ligne 1 « IMT NE - PPI », ligne 2 la température et l'humidité au dixième.

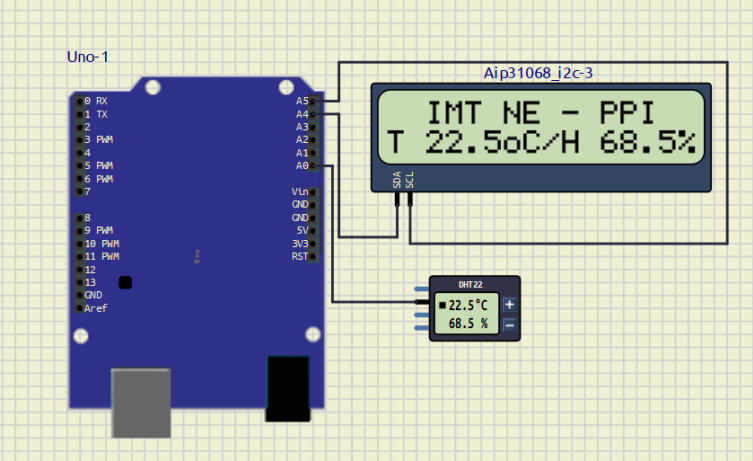


FIGURE 5 – `lcd_i2c_dht22` : mesure du DHT22 affichée sur le LCD

Que peut-on conclure par rapport aux circuits déjà réalisés ?

— **Réutilisation :** chaque circuit précédent a validé un seul élément (LED, capteur, afficheur). Ici on assemble des briques déjà testées, câblage et code, ce qui facilite la recherche d'erreurs.

— **Choix des broches selon le signal :** les LED utilisent des sorties numériques ; le DHT22 envoie un signal numérique sur un seul fil, il peut donc se brancher sur `A0` utilisée comme E/S numérique ; le LCD passe par le bus I2C sur `A4`/`A5`, 2 fils seulement, chaque composant étant identifié par son adresse.

— **Pas de conflit :** les deux composants n'utilisent pas les mêmes broches ; on pourrait ajouter d'autres modules I2C sur les mêmes 2 fils.

1.7 Réalisation du capteur IdO « TempHumiX »

Circuit. Arduino UNO R3, DHT22, LCD 16x2 I2C, bargraphe avec son réseau de résistances de 100 Ω, port série RS232, LED de vie (broche 13), LED Rx/Tx (sur les lignes série) et LED Power, avec leurs résistances de 50 Ω.

Programme d'acquisition. Une machine d'état cadencée par `delay(100)` et un compteur :

— **LED de vie** à 2 Hz : allumée 200 ms sur une période de 500 ms ;

— **toutes les 1 s** : le dateur fait avancer seconde, minute, heure, jour, mois, année (mois de 30 jours), et la date s'affiche sur la ligne 1 du LCD ;

— **toutes les 2 s** : lecture du DHT22, température et humidité au dixième sur la ligne 2 du LCD, et envoi de la date et des mesures sur le port série (COM8).

Contrairement à de longs `delay()`, cette structure ne bloque jamais la boucle : chaque tâche a sa propre période.

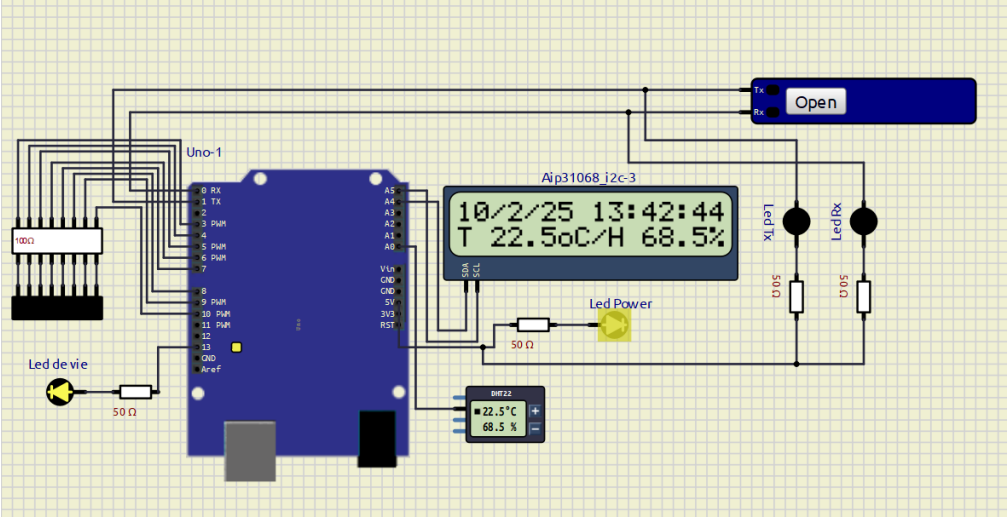


FIGURE 6 – `TempHumiX` : simulation complète du capteur

1.8 Question : Manchester et Manchester différentiel

Ce sont deux **codages en bande de base** (couche physique) : on transforme les bits en niveaux électriques sur le fil. Les deux ont une transition *au milieu de chaque bit*, ce qui permet au récepteur de récupérer l'horloge (auto-synchronisation) et donne un signal de valeur moyenne nulle.

	Manchester	**Manchester différentiel**
Ce qui code le bit	le **sens** de la transition au milieu du bit	la **présence ou non** d'une transition au **début** du bit
Bit 0	front descendant (haut → bas)	transition au début du bit
Bit 1	front montant (bas → haut)	pas de transition au début du bit
Transition au milieu	toujours (donnée)	toujours (horloge seulement)
Fils inversés	tous les bits sont inversés	aucun effet : seul le changement compte
Exemple d'usage	Ethernet 10 Mb/s (10BASE-T)	Token Ring (IEEE 802.5)

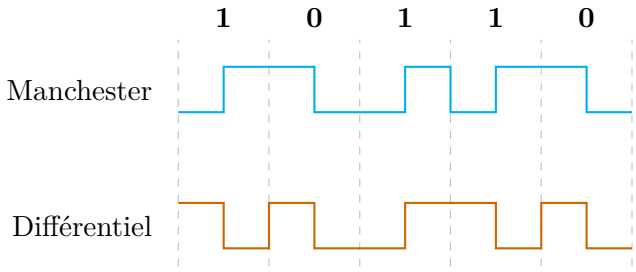


FIGURE 7 – Codage de la suite 10110

À retenir : en Manchester, c'est le *sens* du front qui donne le bit ; en Manchester différentiel, c'est le *changement par rapport au bit précédent*. Le différentiel résiste donc à une inversion des fils (polarité) et au bruit, car on détecte seulement la présence d'une transition. Les deux doublent la fréquence du signal par rapport au débit (2 transitions possibles par bit), donc demandent plus de bande passante qu'un codage NRZ.

2 TP2 – La couche réseau : Wi-Fi et protocole MQTT

Lien avec le cours : niveau 2 (réseau / passerelle) et protocoles de la couche application. MQTT est un protocole *publish/subscribe* au-dessus de TCP/IP : les clients ne se parlent jamais directement, tout passe par le *broker* (Mosquitto). C'est le modèle producteur/consommateur du cours : le capteur produit, plusieurs abonnés consomment.

2.1 Paramètres utilisés

Wi-Fi	SSID `IoT IMT Nord Europe` (2,4 GHz)
Broker	`192.168.1.1:1883`, utilisateur `mos_user`
Client capteur	`IdO_Client_42920`
Publiés	`VERSION_`, `TEMPERATURE_`, `HUMIDITY_`, `GET_DATE_`
Souscrits	`COMMAND_`, `BARGRAPH_`, `BLINK_`

2.2 Prise en main avec MQTTX

Souscrire à la variable d'un autre poste. On reçoit ses messages sans problème. Le broker ne cloisonne pas les postes : tout client authentifié peut lire le topic d'un autre. C'est pratique en TP mais c'est un problème de confidentialité.

Souscrire à `TESTS/#`. `#` est un joker multi-niveaux : on reçoit les messages de *tous* les topics sous `TESTS/`, donc ceux de tous les postes.

Publier avec *Retain*. Le message arrive immédiatement à l'abonnement, alors qu'il a été publié avant. Le broker garde le dernier message retenu du topic et l'envoie à chaque nouvel abonné. Sans Retain, un nouvel abonné ne reçoit rien tant que personne ne republie. Pour l'effacer, on publie un message vide avec Retain coché.

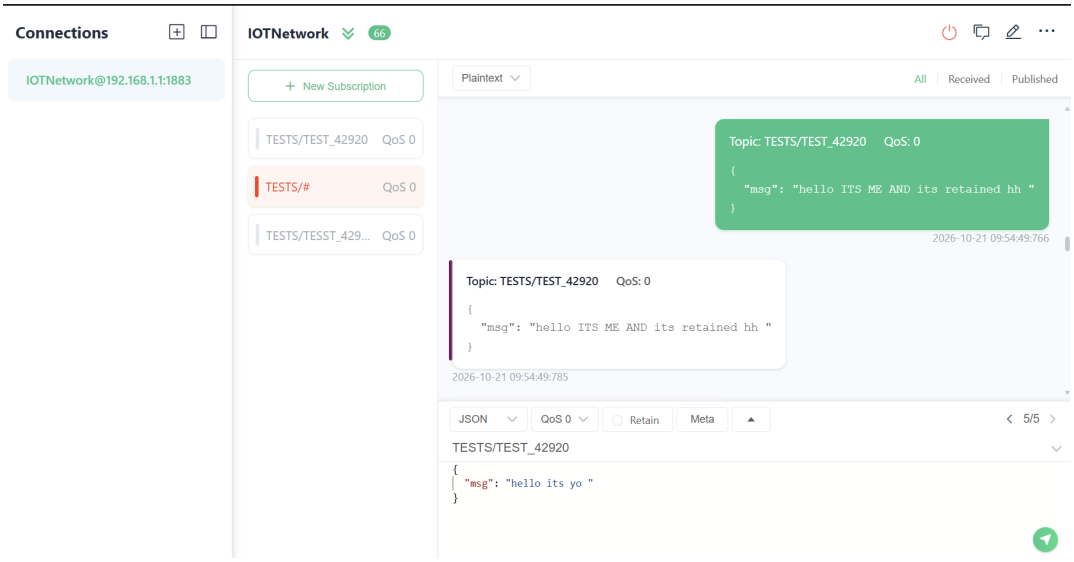


FIGURE 8 – Message retenu reçu dès la souscription (`TESTS/#`)

2.3 Programmation de l'Arduino UNO R4

Passage R3 → R4. Changement de bibliothèque LCD (`Grove_LCD_RGB_Backlight`), suppression du dateur logiciel (remplacé par le RTC interne), boucle cadencée par `delay()`.

Wi-Fi. `setup_wifi()` renvoie `true` si la connexion réussit. Avec le réseau `IoT IMT Nord Europe 5G`, la connexion **échoue** : le module Wi-Fi de l'UNO R4 (ESP32-S3) ne gère que la bande 2,4 GHz.

Il ne voit pas du tout les réseaux 5 GHz, donc `setup_wifi()` renvoie `false` (« WiFi echec » sur le LCD).

NTP + RTC. Au démarrage, l'heure est lue sur le serveur NTP puis copiée dans le RTC. Le code n'est pas bloquant : en cas d'échec, le LCD affiche « Pas de synchro » et la mesure continue.



FIGURE 9 – Le boîtier TempHumiX réel : date NTP et mesure sur le LCD, bargraphe piloté par MQTT

Publication. `setup_mqtt()` renvoie 0 si OK, sinon le code d'erreur MQTT. Les 4 topics sont publiés toutes les 2 s.

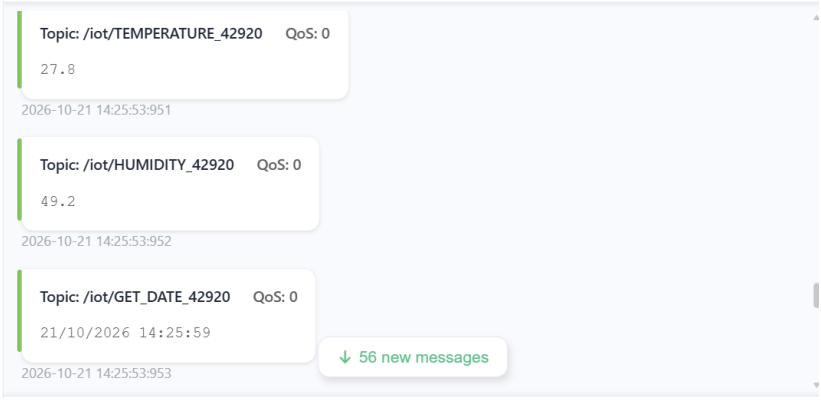


FIGURE 10 – Valeurs publiées par le capteur, vues dans MQTTX

Codes commande : 10 = synchro NTP, 20/30 = démarrage/arrêt mesure, 40/50/60 = clignotement lent/rapide/personnalisé.

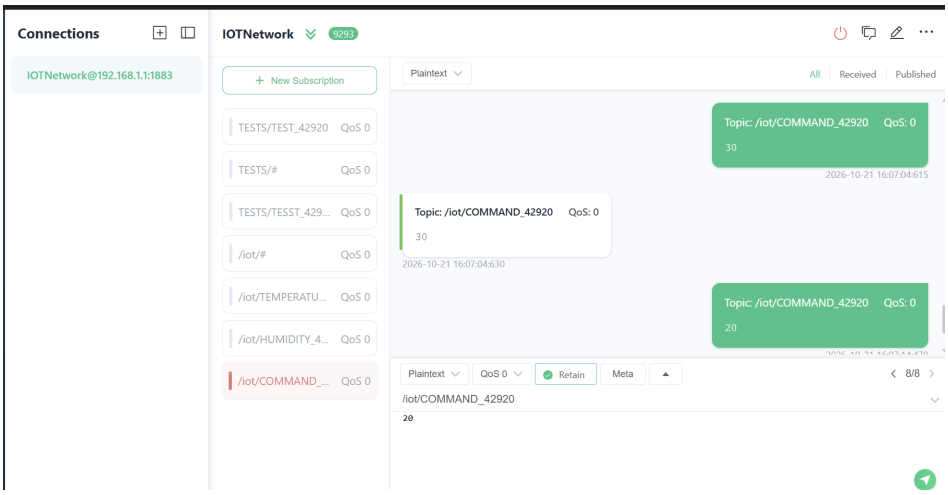


FIGURE 11 – Envoi des commandes 30 puis 20 depuis MQTTX

2.4 Amélioration de la communication

Avant : 4 topics toutes les 2 s même sans changement, soit environ 120 messages par minute, la plupart identiques.

a) La version ne change jamais : elle n'est publiée qu'après une synchronisation NTP (commande 10).

b) Température, humidité et date ne sont publiées que si T ou H a changé *au dixième près* (la précision publiée).

```
if ((int)(t*10) != (int)(ancienT*10) || (int)(h*10) != (int)(ancienH*10)) {
  publier_mqtt(); ancienT = t; ancienH = h;
}
```

Intérêt : moins de trafic sur le Wi-Fi partagé, moins de travail pour le broker et les abonnés, et moins d'énergie (l'émission radio est ce qui consomme le plus). C'est l'esprit événementiel de MQTT. **Limite :** si la température reste stable, un nouvel abonné ne reçoit rien. On peut corriger cela en publiant avec Retain.

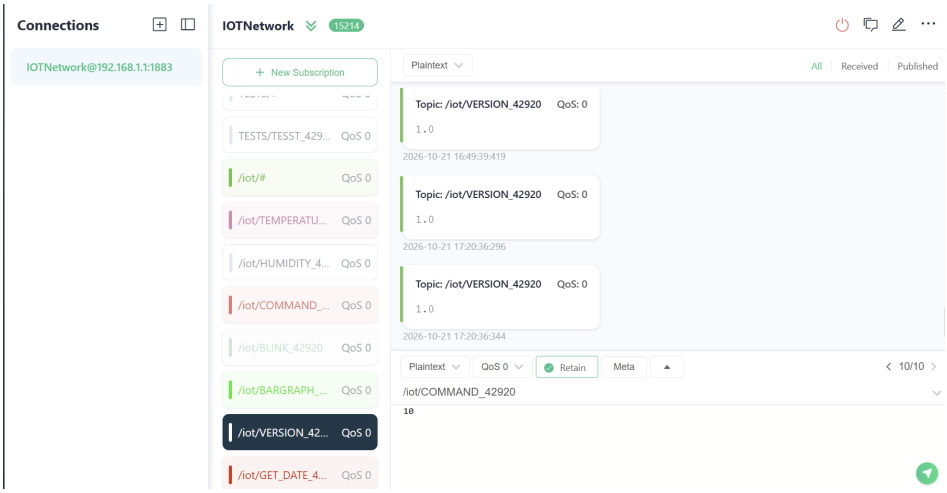


FIGURE 12 – La version n'est publiée qu'à la réception de la commande 10

2.5 Analyse des trames avec Wireshark

Capture avec le filtre `tcp port 1883`, puis affichage filtré sur `mqtt`. Le PC (client) est en `192.168.1.168`, le broker en `192.168.1.1`.

Paquet	Sens	Champs relevés
CONNECT	client → broker	Client ID `mqttx_258ef9b4`, Keep Alive 60 s, User/Password `mos_user`, MQTT v5.0
CONNACK	broker → client	Return code 0 (connexion acceptée)
SUBSCRIBE	client → broker	Topic filter + QoS demandé
SUBACK	broker → client	*Granted QoS 0*
PUBLISH	deux sens	Topic, message, QoS 0, Retain non, DUP non
PINGREQ/RESP	client ↔ broker	56 octets, en l'absence d'autre trafic
DISCONNECT	client → broker	58 octets, fin volontaire de la session

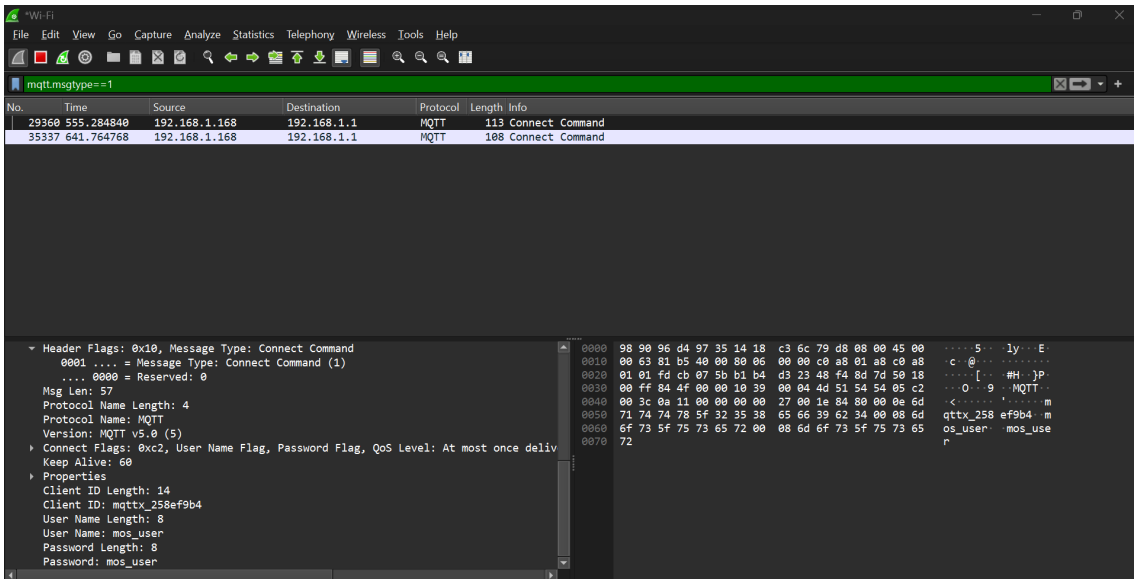


FIGURE 13 – Paquet CONNECT : l'utilisateur et le mot de passe circulent en clair

Où apparaît le mot de passe ? Dans le paquet CONNECT, champs *User Name* et *Password*, lisible en clair jusque dans la vue hexadécimale. Le port 1883 n'est **pas chiffré** : toute personne sur le même Wi-Fi peut récupérer les identifiants. Il faudrait passer en MQTT sur TLS (port 8883).

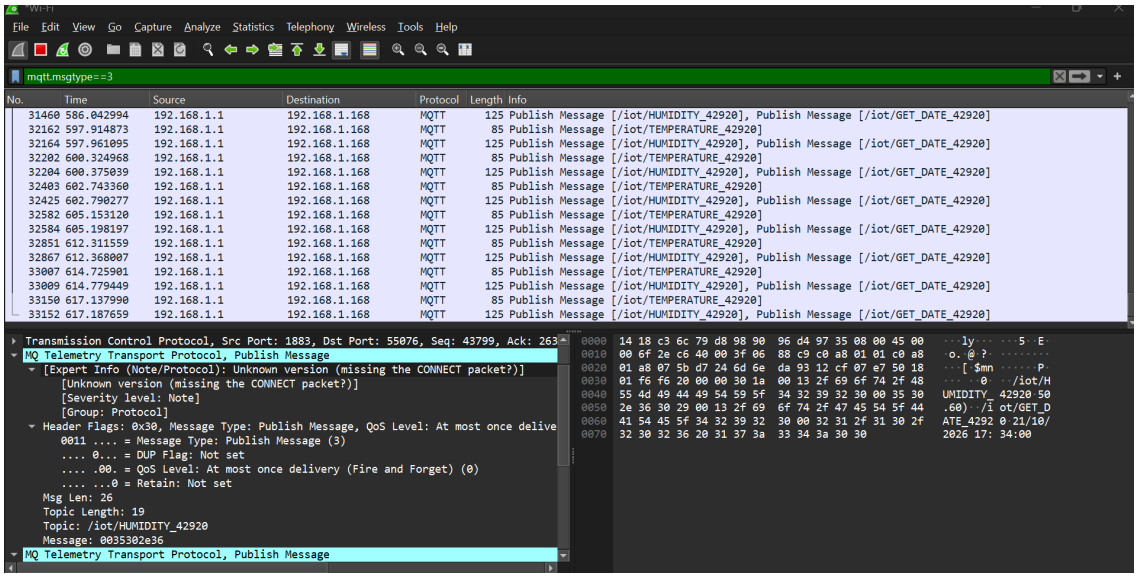


FIGURE 14 – Paquets PUBLISH du capteur (85 et 125 octets)

Taille d'un PUBLISH de température. 85 octets sur le réseau, dont seulement une trentaine d'octets MQTT (en-tête fixe de 2 octets, topic `/iot/TEMPERATURE_42920`, valeur `27.8`) ; le reste est l'en-tête Ethernet/IP/TCP. Une requête HTTP équivalente (`POST` + en-têtes `Host`, `Content-Type`, `Content-Length`...) dépasse facilement 200 à 300 octets, sans compter l'ouverture d'une connexion TCP à chaque requête. MQTT garde une seule connexion ouverte et est environ 10 fois plus léger.

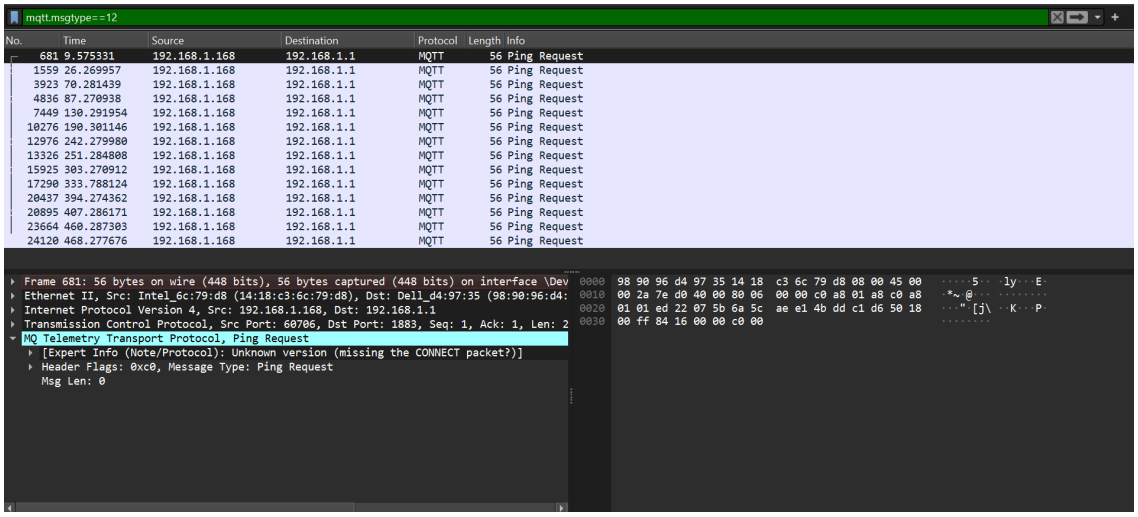


FIGURE 15 – PINGREQ envoyés par le client pour maintenir la session

Keep Alive. C'est la durée maximale (60 s ici) sans échange entre client et broker. Si rien n'est envoyé pendant cette durée, le client émet un PINGREQ et le broker répond PINGRESP. Si le broker ne reçoit rien pendant 1,5 fois le Keep Alive, il considère le client mort, ferme la connexion et publie son éventuel *Last Will*.

2.6 QoS, messages retenus et jokers

— **QoS 0** : un seul PUBLISH, pas d'acquittement.

— **QoS 1** : PUBLISH + PUBACK (doublons possibles).

— **QoS 2** : 4 paquets PUBLISH, PUBREC, PUBREL, PUBCOMP avec le même identifiant de

message : livraison exactement une fois.

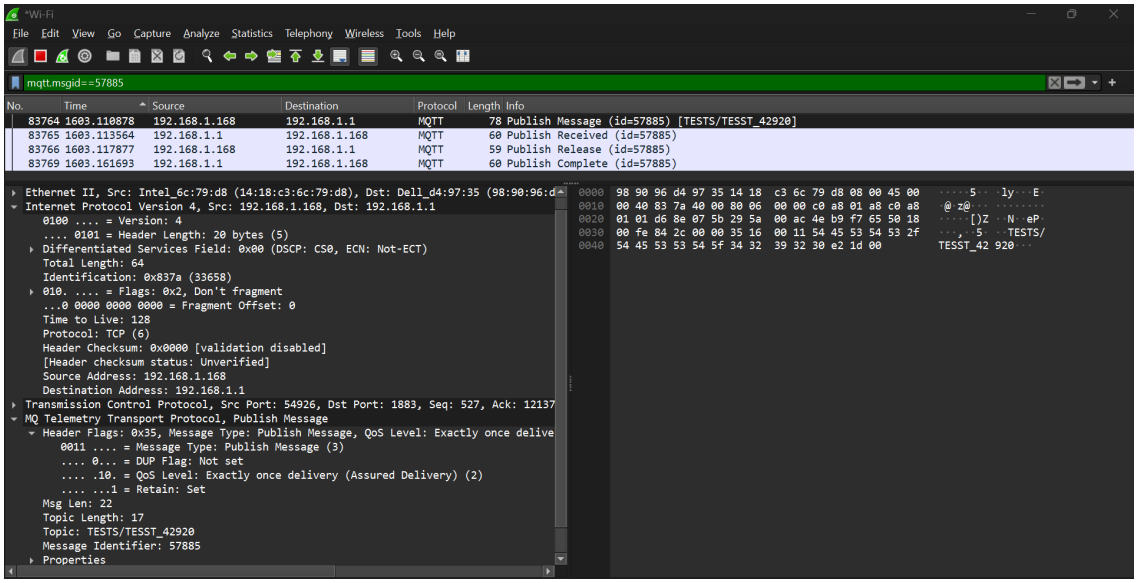


FIGURE 16 – Échange QoS 2 (id 57885) : PUBLISH, PUBREC, PUBREL, PUBCOMP

Jokers : `TESTS/+` = un seul niveau (`TESTS/TEST_42920` mais pas `TESTS/a/b`) ; `TESTS/#` = tous les niveaux ; `$SYS/#` = topics internes du broker (clients connectés, messages reçus…), selon les droits donnés par l'ACL.

2.7 Questions générales

1. Acheminement de l'information (modèle OSI / TCP-IP).

Couche	Dans notre réseau
Application (7)	MQTT (publish/subscribe), NTP pour l'heure
Transport (4)	TCP port 1883 (MQTT), UDP port 123 (NTP)
Réseau (3)	IPv4 `192.168.1.0/24` : capteur et PC en DHCP, broker en `192.168.1.1`
Liaison (2)	Wi-Fi 802.11 côté capteur, Ethernet côté broker ; adresses MAC
Physique (1)	Ondes radio 2,4 GHz, câble cuivre

Le capteur encapsule sa mesure dans MQTT → TCP → IP → trame Wi-Fi. Le RUTX50 (point d'accès) reçoit la trame radio et la renvoie en Ethernet vers le broker : dans Wireshark, l'adresse MAC de `192.168.1.1` est celle d'une machine Dell, le broker tourne donc sur un serveur distinct du routeur. Le broker désencapsule jusqu'à MQTT puis republie vers chaque abonné par le chemin inverse.

2. Services applicatifs du Teltonika RUTX50. Serveur DHCP, DNS, NTP, pare-feu, VPN (OpenVPN, IPsec, WireGuard), interface Web/SSH, SNMP, broker et client MQTT, Modbus TCP, envoi de données vers un serveur, gestion à distance (RMS). Dans l'architecture IoT, ces services se situent au niveau **Edge/Fog** : traitement près des capteurs avant le Cloud. Le routeur peut jouer plusieurs rôles :

— **pont** (couche 2) entre le Wi-Fi et l'Ethernet du même réseau local ;

— **routeur** (couche 3) entre le LAN IoT et le WAN (4G/5G) ;

— **passerelle** (jusqu'à la couche 7) quand il convertit un protocole en un autre, par exemple Modbus vers MQTT.

3. MCU ou MPU ? Un **MPU** : processeur ARM Cortex-A7 quatre cœurs (Qualcomm IPQ4018, 717 MHz) avec RAM et flash externes. Il exécute un vrai système d'exploitation, **RutOS**, basé sur Linux OpenWrt. Un MCU ne pourrait pas faire tourner autant de services en parallèle.

4. Connexions Wi-Fi illimitées ? Non. Le constructeur annonce environ 150 clients simultanés. Le temps d'antenne est partagé (accès CSMA/CA) et la mémoire, le CPU et la plage DHCP sont limités. Risques : un mot de passe Wi-Fi unique et partagé (connu de tous), et tous les appareils sur le même réseau peuvent écouter le trafic non chiffré (MQTT en clair) ou saturer le point d'accès. La topologie est en **étoile** : chaque objet est relié au point d'accès central.

5. Mesures de sécurité du firmware. WPA2/WPA3, pare-feu activé par défaut, changement obligatoire du mot de passe par défaut à la première connexion, accès d'administration en HTTPS/SSH, blocage après plusieurs tentatives de connexion ratées, VPN, VLAN et isolation des clients Wi-Fi, mises à jour du firmware à distance (FOTA via RMS). *(Liste à confronter aux notes de version de la version installée sur le routeur.)*

6. Commentaire des logs Mosquitto.

— Mosquitto 2.1.2 démarre et charge `mosquitto.conf` ; les modules bridge, persistance, TLS, TLS-PSK et WebSockets sont disponibles.

— Le service tourne sous l'utilisateur `mosquitto` et non `root` : bonne pratique (moindre privilège).

— « Restored 0 . . . » : aucune donnée persistée à restaurer (premier démarrage ou base vide).

— **Avertissements** : les fichiers `passwd` et `acl` sont lisibles par tout le monde et n'appartiennent pas à `mosquitto`. Les futures versions refuseront de les charger. Correction : `chmod 0700` et `chown mosquitto`.

— Le pattern ACL `TESTS/#` ne contient ni `%c` (client ID) ni `%u` (utilisateur) : la règle est la même pour tous, chaque utilisateur peut lire et écrire les topics des autres. C'est ce qu'on a observé avec MQTTX.

— Le plugin de sécurité gère l'authentification (`basic-auth`) et le contrôle d'accès (`acl-check`).

— Écoute en IPv4 et IPv6 sur le port 1883 : port **non chiffré** ; TLS est disponible mais pas utilisé.

3 TP3 – La couche application : IHM Python

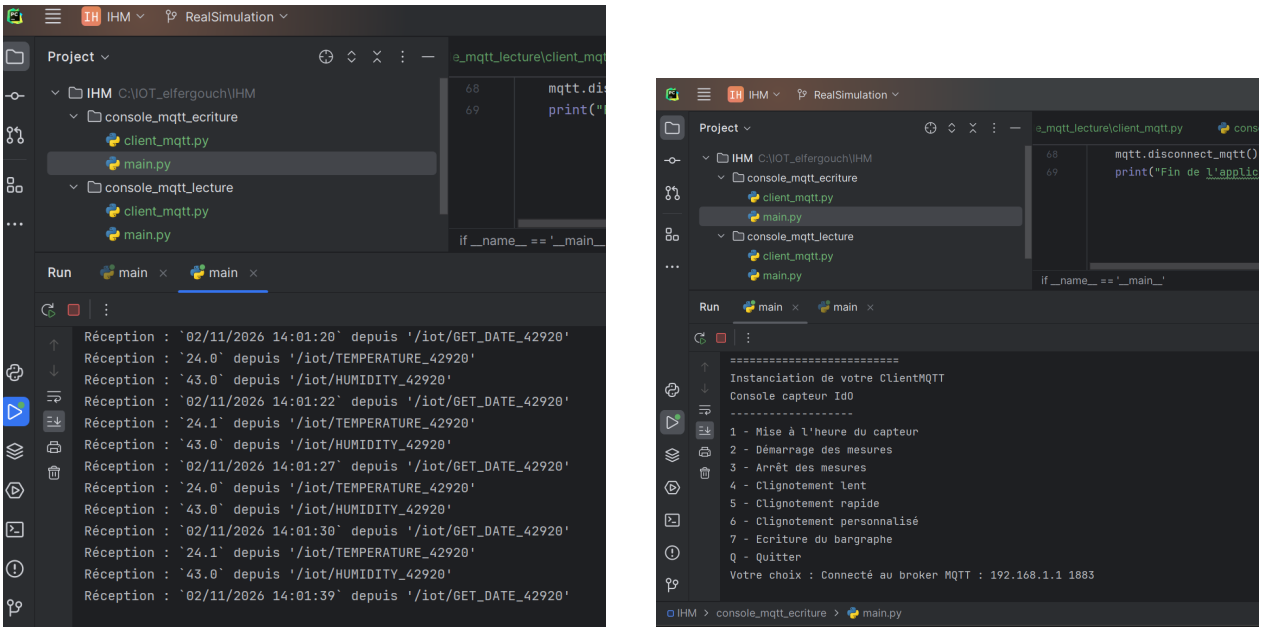
Lien avec le cours : niveau 3 (application) et éléments de la plateforme IdO du niveau 4 : connectivité (client MQTT), traitement, **stockage** (SQLite) et **visualisation** (courbes). Le PC est un client MQTT comme le capteur : il souscrit aux mesures et publie les commandes.

3.1 Client MQTT en console

Une classe `ClientMQTT` (bibliothèque `paho-mqtt`) regroupe `connect_mqtt`, `disconnect_mqtt`, `publish_mqtt`, `subscribe_mqtt` et `wait_mqtt`.

— **Lecture** : souscription aux 4 topics du capteur puis `wait_mqtt(0)` (`loop_forever`).

— **Écriture** : menu console, chaque choix publie un code sur `/iot/COMMAND_42920` ou une valeur sur `BARGRAPH/BLINK`. L'arrêt des mesures envoie 30 puis 40 (clignotement lent), le démarrage envoie 20 puis 50 (clignotement rapide) : l'état se voit sur la LED de vie.



(a) Client en lecture (b) Client en écriture (menu)

FIGURE 17 – Les deux clients console exécutés en parallèle

Exécution en parallèle. Deux clients ne peuvent pas avoir le même identifiant : le broker déconnecterait le premier. Le client d'écriture s'appelle donc `Client_E42920`.

3.2 Thread MQTT

`loop_forever()` bloque le programme : impossible de lire le menu en même temps. La lecture est donc déplacée dans un `ThreadMQTT` qui appelle `wait_mqtt(timeout=1.0)` en boucle et range les valeurs reçues dans les attributs du client (`temperature`, `humidite`, ...). Le menu reste disponible et le choix 8 affiche les dernières valeurs.

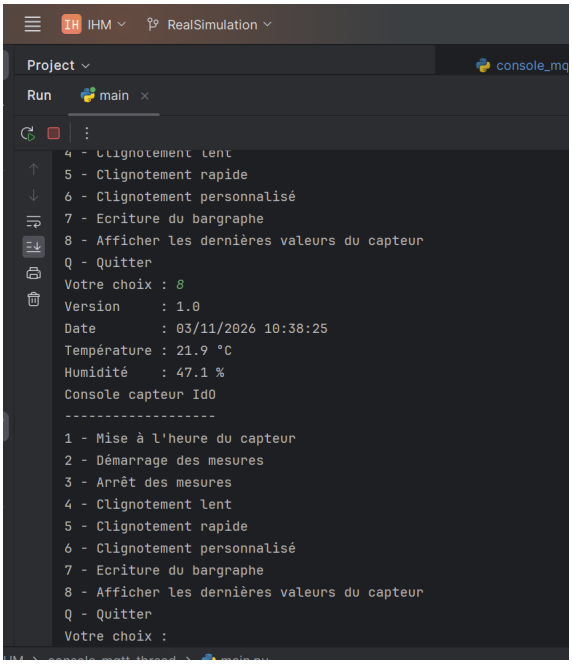


FIGURE 18 – Lecture en arrière-plan + menu de commandes dans un même programme

3.3 IHM PySide6 – notice de fonctionnement

L'interface est dessinée avec `pyside6-designer` (`mon_ihm.ui`) puis convertie en Python avec `pyside6-uic`.



FIGURE 19 – Fenêtre principale de l'IHM

1. **Connexion** : vérifier l'adresse IP, le port, l'utilisateur et le mot de passe (pré-remplis, avec contrôle de saisie), puis cliquer sur *Connecter*. Le thread MQTT démarre et un timer rafraîchit l'affichage toutes les 1000 ms.
2. **Affichage** : date/heure du capteur, température et humidité.
3. **Commandes** : synchro NTP (10), arrêter/démarrer les mesures (30/20), clignotement lent/rapide (40/50).
4. **Clignotement personnalisé** : saisir la période en ms (100 à 10000) puis cliquer : la période part sur `BLINK`, puis la commande 60.
5. **Bargraphe** : saisir une valeur de 0 à 1023 puis *Entrée*.
6. **Courbes** : menu *Fichier*, ouvre la fenêtre des courbes.
7. **Déconnexion** : arrête le timer et le thread. Les boutons de commande sont grisés tant qu'on n'est pas connecté.

3.4 Archivage SQLite

La base `IOT.db` contient une table `IOT_Data(Id_capteur, Date, Version, Temperature, Humidite)`. La connexion à la base est ouverte *dans* le thread MQTT (SQLite n'accepte pas qu'un objet de connexion soit utilisé depuis un autre thread). Une ligne est insérée à la réception de `GET_DATE`, dernier topic publié par l'Arduino : température et humidité sont alors à jour. La date est convertie au format `AAAA-MM-JJ hh:mm:ss` pour que le tri et les requêtes `BETWEEN` fonctionnent.

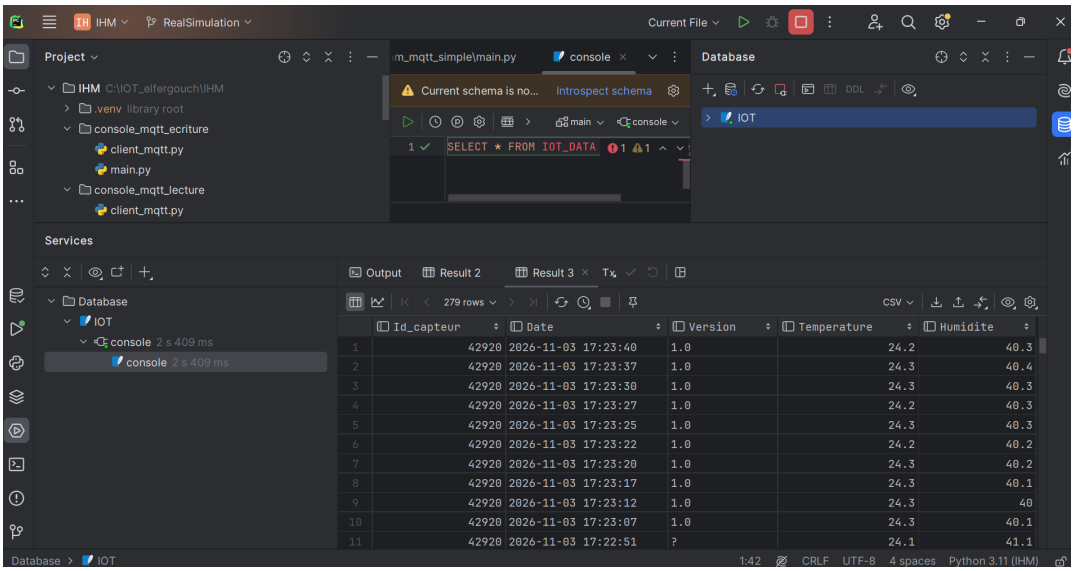


FIGURE 20 – Contenu de la table `IOT_Data` (279 mesures)

3.5 Affichage des courbes

La boîte de dialogue `CourbeDialog` lit les mesures entre deux dates (`SELECT ... WHERE Date BETWEEN ? AND ?`) et trace température et humidité avec Matplotlib. La figure est convertie en image et affichée dans un `QGraphicsView`.

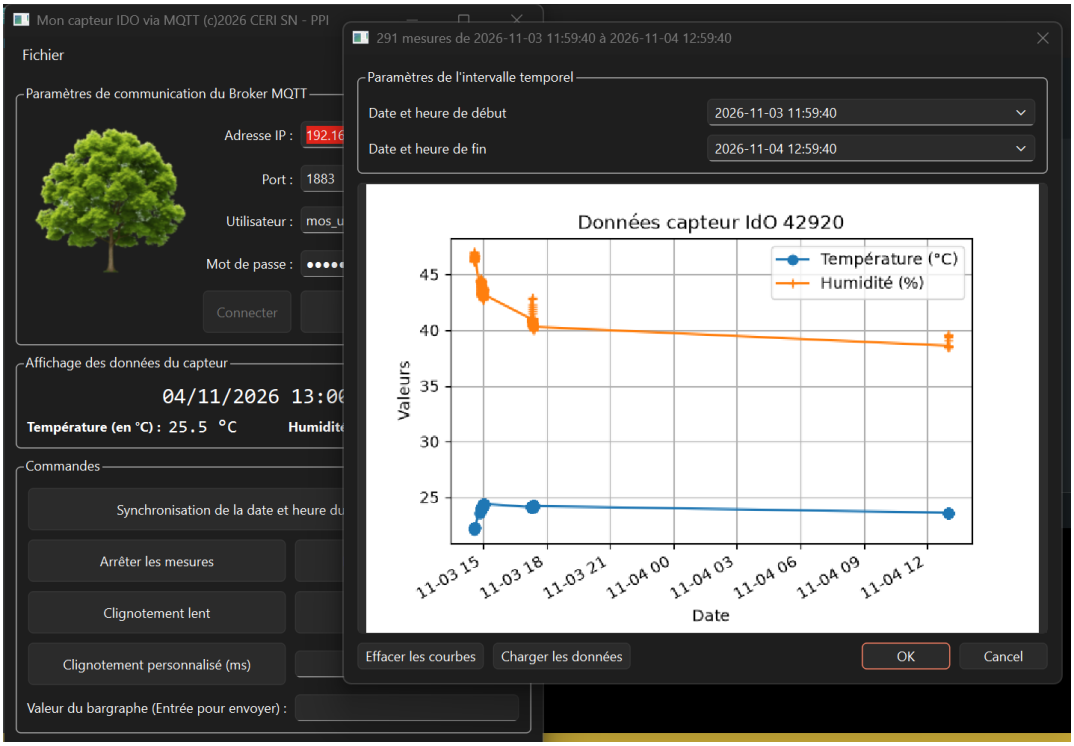


FIGURE 21 – Courbes de température et d'humidité sur 24 h (291 mesures)

4 Pour aller plus loin : interface web

Lien avec le cours : même rôle que l'IHM PySide6 (niveau application), mais accessible depuis n'importe quel navigateur du réseau, sans installer Python sur le poste.

L'interface reprend les fonctions de l'IHM : connexion au broker, affichage en direct, commandes du capteur, consultation de la base et courbes.

Architecture.

— **Serveur Node.js / Express** : il est le client MQTT (`Client_W42920`, pour pouvoir tourner en même temps que l'IHM Python). Il souscrit aux 4 topics du capteur, publie les commandes et archive chaque mesure dans SQLite à la réception de `GET_DATE`, comme l'IHM Python (même table `IOT_Data`).

— **API REST** : une route par action (connexion, commandes, bargraphe, clignotement, lecture des mesures).

— **Temps réel** : les nouvelles valeurs sont poussées au navigateur par *Server-Sent Events*, sans recharger la page.

— **Front** : HTML et JavaScript natif, courbes avec Chart.js.

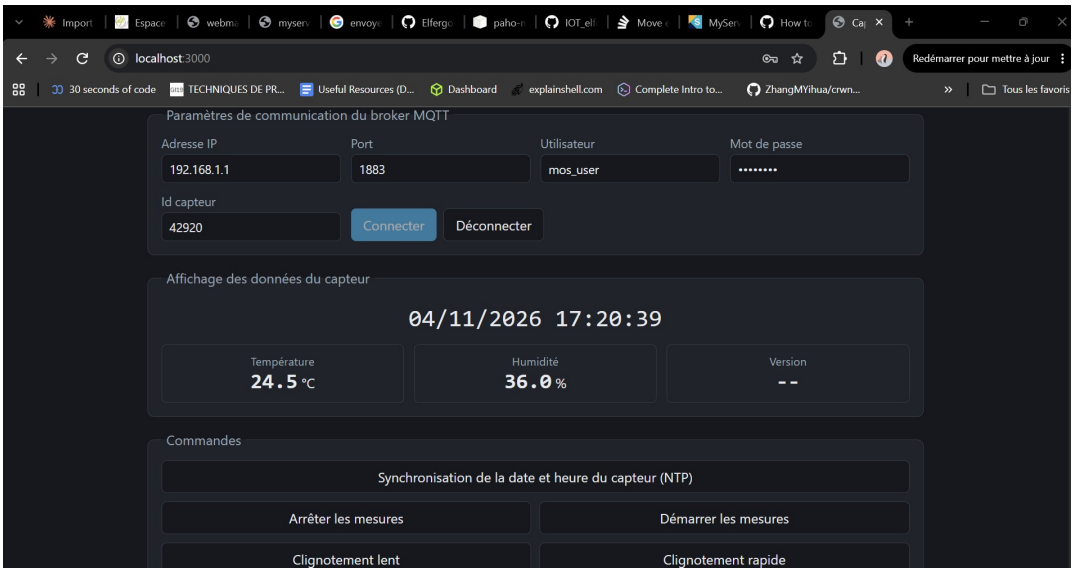


FIGURE 22 – Tableau de bord : paramètres du broker, mesures en direct et commandes

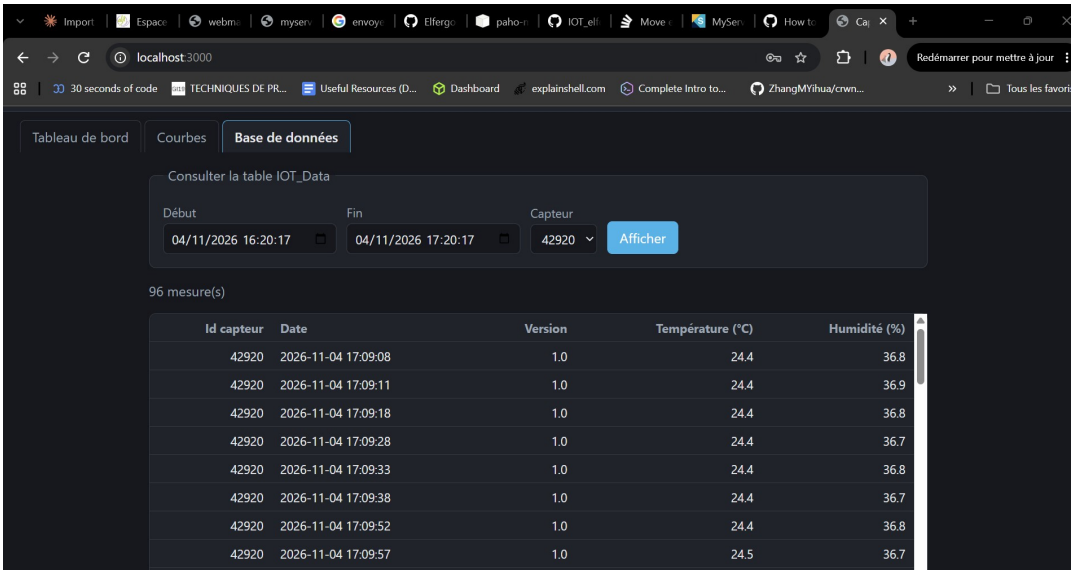


FIGURE 23 – Consultation de la table `IOT_Data` entre deux dates

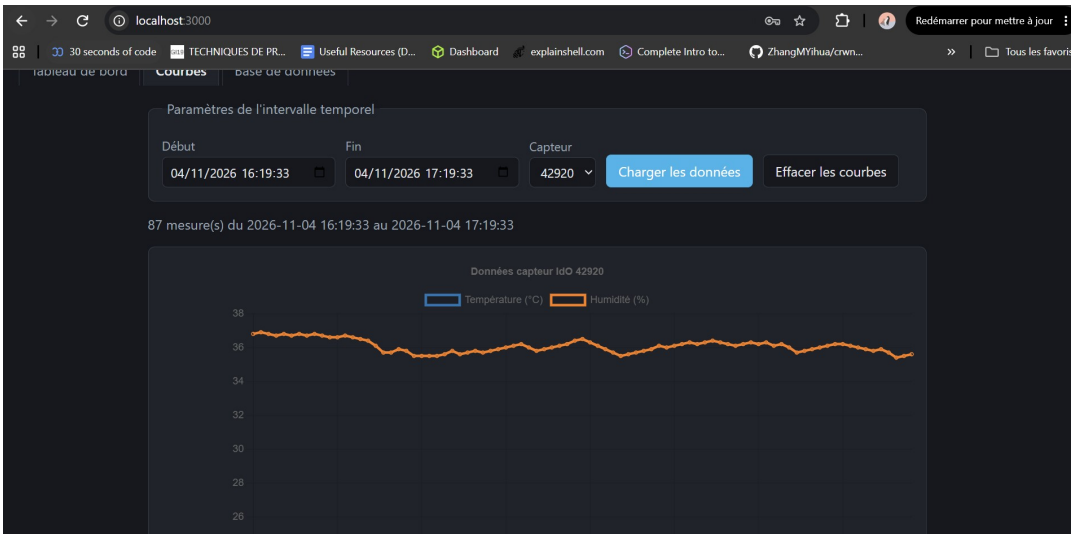


FIGURE 24 – Courbes sur la dernière heure de données

Conclusion

Acquis.

— Une chaîne IoT complète, du capteur à la base de données, en suivant les niveaux de l'architecture du cours.

— Programmation embarquée non bloquante (boucle cadencée, RTC, NTP).

— Le modèle publish/subscribe de MQTT, ses QoS, le Retain et les jokers, vérifiés trame par trame dans Wireshark.

— Une IHM Python multi-thread avec archivage et visualisation, puis sa version web.

Difficultés.

— Le réseau 5 GHz invisible pour l'UNO R4 (bande 2,4 GHz uniquement).

— Le serveur NTP : il a fallu utiliser l'adresse disponible sur le réseau IoT.

— SQLite et les threads : la connexion doit être créée dans le thread qui l'utilise.

— Le format de date : `JJ/MM/AAAA` ne se trie pas, conversion en ISO.

Perspectives.

— Sécurité : MQTT sur TLS (8883), un compte et une ACL par capteur (`%u`/`%c`).

— Challenge : sérialiser les mesures dans un seul topic `DATA_` et afficher tous les capteurs de la salle (souscription `/iot/#`).

— Publier les mesures avec Retain pour qu'un nouvel abonné ait tout de suite la dernière valeur.

— Faire évoluer l'interface web : authentification, plusieurs capteurs, accès depuis l'extérieur.